Notas: Categorías, Familias, Combinaciones y Aplicación a IA

Octavio

15 de febrero de 2026

Índice general

Capítulo 1

Categorías base

1.1. La categoría Ab: grupos abelianos

La categoría **Ab** se define así:

- **Objetos:** grupos abelianos $(A, +, 0, -)$.
- **Morfismos:** homomorfismos de grupos $f : A \to B$.
- **Identidad:** $\mathrm{id}_A : A \to A$.
- **Composición:** si $f : A \to B$ y $g : B \to C$, entonces $g \circ f : A \to C$.

1.2. La categoría Top: espacios topológicos

La categoría **Top** se define así:

- **Objetos:** espacios topológicos (X, τ).
- **Morfismos:** funciones continuas $g : X \to Y$.
- **Identidad y composición:** las usuales de funciones.

Capítulo 2

Categorías de familias

2.1. Familias con índice fijo: $\mathcal{C}^I$

Sea $\mathcal{C}$ una categoría y sea I un conjunto. Considera I como *categoría discreta* (solo identidades). Entonces:

$$\mathcal{C}^I \;:=\; \mathrm{Fun}(I_{\mathrm{disc}}, \mathcal{C}).$$

Equivalentemente:

- **Objeto:** una familia $(A_i)_{i\in I}$ con $A_i \in \mathrm{Ob}(\mathcal{C})$.
- **Morfismo:** $(f_i)_{i\in I} : (A_i) \to (B_i)$ con $f_i : A_i \to B_i$ en $\mathcal{C}$.
- **Identidad:** $(\mathrm{id}_{A_i})_{i\in I}$.
- **Composición:** $(g_i) \circ (f_i) = (g_i \circ f_i)_{i\in I}$.

2.2. Familias con índice variable: $\mathrm{Fam}(\mathcal{C})$

La categoría $\mathrm{Fam}(\mathcal{C})$ se define así:

- **Objeto:** un par (I, A) donde I es un conjunto y $A : I \to \mathrm{Ob}(\mathcal{C})$, es decir, una familia $(A_i)_{i\in I}$ de objetos de $\mathcal{C}$.
- **Morfismo:** de (I, A) a (J, B) es un par

  $$(u, (f_i)_{i\in I})$$

  donde $u : I \to J$ es una función y, para cada $i \in I$,

  $$f_i : A_i \longrightarrow B_{u(i)}$$

  es un morfismo en $\mathcal{C}$.

La identidad y composición son:

$$\mathrm{id}_{(I,A)} = \Big(\mathrm{id}_I,\ (\mathrm{id}_{A_i})_{i\in I}\Big),$$

y si $(u, f) : (I, A) \to (J, B)$ y $(v, g) : (J, B) \to (K, C)$, entonces

$$(v, g) \circ (u, f) = \Big(v \circ u,\ (g_{u(i)} \circ f_i)_{i\in I}\Big).$$

2.3. Ejemplo: $\mathrm{Fam}(\mathbf{Ab})$

Un objeto de $\mathrm{Fam}(\mathbf{Ab})$ es una familia de grupos abelianos $(A_i)_{i\in I}$. Un morfismo incluye una función $u : I \to J$ y homomorfismos $f_i : A_i \to B_{u(i)}$.

Capítulo 3

Formas de combinar Ab y Top

3.1. Producto de categorías: Ab $\times$ Top

La categoría producto **Ab** $\times$ **Top** se define así:

- **Objeto:** un par (A, X) con $A \in$ **Ab** y $X \in$ **Top**.
- **Morfismo:** $(f, g) : (A, X) \to (B, Y)$ con $f : A \to B$ en **Ab** y $g : X \to Y$ en **Top**.
- **Identidad:** $(\mathrm{id}_A, \mathrm{id}_X)$.
- **Composición:** $(f_2, g_2) \circ (f_1, g_1) = (f_2 \circ f_1,\ g_2 \circ g_1)$.

Aquí *no* se exige compatibilidad entre la estructura abeliana y la topológica.

3.2. Objetos con ambas estructuras compatibles: TopAb

Definimos **TopAb** como la categoría de *grupos abelianos topológicos*.

- **Objeto:** un espacio topológico A con mapas continuos

$$+ : A \times A \to A, \qquad - : A \to A, \qquad 0 : * \to A$$

  tales que $(A, +, 0, -)$ es un grupo abeliano.

- **Morfismo:** una función continua $f : A \to B$ tal que

$$f(x + y) = f(x) + f(y), \qquad f(0) = 0, \qquad f(-x) = -f(x).$$

Equivalente: **TopAb** son los *objetos abelianos internos* en **Top**.

3.3. Plantilla para diseñar una categoría "mezclada"

Para construir una categoría de objetos con estructura adicional:

1. Elige una categoría ambiente $\mathcal{E}$ (por ejemplo **Top**).
2. Especifica los **datos** (operaciones) como morfismos en $\mathcal{E}$.
3. Impone los **axiomas** como diagramas conmutativos (asociatividad, identidad, inversos, conmutatividad, etc.).
4. Define los **morfismos** como los que preservan esa estructura.
5. Verifica identidad y composición: preservan la estructura automáticamente.

Capítulo 4

Functores entre Ab y Top

4.1. Qué es un funtor

Un funtor $F : \mathcal{C} \to \mathcal{D}$ consiste de:

- asignación en objetos: $A \mapsto F(A)$,
- asignación en morfismos: $f : A \to B \mapsto F(f) : F(A) \to F(B)$,

tal que:

$$F(\mathrm{id}_A) = \mathrm{id}_{F(A)}, \qquad F(g \circ f) = F(g) \circ F(f).$$

4.2. Funtor discreto: $D : \mathbf{Ab} \to \mathbf{Top}$

Definimos D por:

$$D(A) = A \text{ con topología discreta}, \qquad D(f) = f \text{ (misma función)}.$$

Entonces D es un funtor porque preserva identidades y composición.

4.3. Funtor de olvido: $U : \mathbf{TopAb} \to \mathbf{Ab}$

Definimos U por:

$$U(A, \tau, +, 0, -) = (A, +, 0, -), \qquad U(f) = f.$$

Es un funtor que *olvida* la topología.

4.4. Homología singular: $H_n : \mathbf{Top} \to \mathbf{Ab}$

Para cada $n \geq 0$ existe un funtor (invariante) llamado homología singular:

$$H_n : \mathbf{Top} \to \mathbf{Ab},$$

que asigna a un espacio topológico X un grupo abeliano $H_n(X)$, y a una función continua $g : X \to Y$ un homomorfismo inducido

$$H_n(g) : H_n(X) \to H_n(Y),$$

cumpliendo:

$$H_n(\mathrm{id}_X) = \mathrm{id}_{H_n(X)}, \qquad H_n(g \circ f) = H_n(g) \circ H_n(f).$$

Capítulo 5

Coproductos y suma directa desde familias

5.1. Suma directa desde familias en Ab

Como **Ab** tiene coproductos (sumas directas), existe un funtor:

$$\Sigma : \mathrm{Fam}(\mathbf{Ab}) \to \mathbf{Ab}, \qquad \Sigma(I, (A_i)) = \bigoplus_{i \in I} A_i.$$

Este funtor "colapsa" una familia de grupos abelianos a su suma directa.

Observación 5.1. En práctica, $\bigoplus_{i \in I} A_i$ es el coproducto en **Ab**: elementos con soporte finito sobre el índice I.

Capítulo 6

Abstracción categórica aplicada a cálculos de IA

6.1. Motivación: separar estructura de contenido

En IA es común construir *pipelines* de transformación y modelos compuestos. La abstracción categórica separa:

- **Estructura:** composición, productos/coprodcutos, invariancias.
- **Contenido:** datos concretos y parámetros entrenables.

6.2. Ventajas concretas

- **Composición = pipelines.** Cada etapa es un morfismo; el pipeline es composición.
- **Modularidad.** Cambias un bloque conservando dominio/codominio y todo sigue bien definido.
- **Reuso y pruebas.** Diagramas conmutativos funcionan como pruebas de coherencia.
- **Optimización algebraica.** Reasociar/fusionar cómputos sin cambiar resultados.
- **Batching como familias.** Un mini-batch se modela en $\mathcal{C}^I$.
- **Invariancias explícitas.** Simetrías como acciones de grupo reducen complejidad efectiva.
- **Interoperabilidad.** Functores conectan representaciones (imagen→grafo→TDA→vector).

6.3. Ejemplo 1: Pipeline de IA como composición

Define una categoría (conceptual) $\mathcal{P}$:

- **Objetos:** espacios de datos (Raw, Tokens, Embeddings, Logits, ...).

- **Morfismos:** transformaciones computables (tokenizar, normalizar, embed, clasificar).

  Un pipeline típico:

$$\text{Raw} \xrightarrow{\text{tokenize}} \text{Tokens} \xrightarrow{\text{embed}} \text{Emb} \xrightarrow{\text{head}} \text{Logits}.$$

El modelo completo es el morfismo compuesto:

$$\text{head} \circ \text{embed} \circ \text{tokenize} : \text{Raw} \to \text{Logits}.$$

6.4. Ejemplo 2: Mini-batching como familia

Sea $\mathcal{C}$ una categoría (p.ej. **Vect**) e I un conjunto índice. Entonces:

$$\mathcal{C}^I = \text{Fun}(I_{\text{disc}}, \mathcal{C}).$$

Un batch de embeddings:

$$(E_i)_{i \in I} \in \mathbf{Vect}^I.$$

Aplicar una transformación por componente:

$$(f_i : E_i \to E_i')_{i \in I} : (E_i)_{i \in I} \to (E_i')_{i \in I},$$

y composición por componente:

$$(g_i) \circ (f_i) = (g_i \circ f_i)_{i \in I}.$$

6.5. Ejemplo 3: TDA como cadena de traductores (funtores)

Esquema conceptual:

$$\text{Datos} \longrightarrow \text{Complejos simpliciales} \longrightarrow \text{Módulos de persistencia} \longrightarrow \mathbf{Vect}.$$

Más explícito:

$$(X, d) \xmapsto{\text{VR}_\bullet} \{\text{VR}_\epsilon(X)\}_{\epsilon \geq 0} \xmapsto{H_k} \{H_k(\text{VR}_\epsilon(X))\}_{\epsilon \geq 0} \xmapsto{\Phi} \mathbb{R}^m.$$

6.6. Ejemplo 4: Backprop y composición (lectura estructural)

En entrenamiento:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}.$$

Si el modelo es una composición:

$$f = f_n \circ f_{n-1} \circ \cdots \circ f_1,$$

la regla de la cadena expresa compatibilidad con la composición:

$$D(f_n \circ \cdots \circ f_1) = Df_n \circ \cdots \circ Df_1,$$

con las composiciones adecuadas (jacobianos/diferenciales).

6.7. Reglas prácticas: tamaño y control de combinaciones

No hay un límite universal de combinaciones, pero sí restricciones por *tamaño*.

6.7.1. Categorías grandes y familias pequeñas

Permitir índices arbitrarios en $\mathrm{Fam}(\mathcal{C})$ puede producir una categoría muy grande. Para controlar esto se usan restricciones típicas:

- $\mathrm{Fam}_{\mathrm{fin}}(\mathcal{C})$: solo familias con índice finito.
- $\mathrm{Fam}_{\mathbb{N}}(\mathcal{C})$: solo familias numerables.
- $\mathrm{Fam}_{U}(\mathcal{C})$: índices "pequeños" respecto a un universo de Grothendieck U.

Notas finales

Si quieres, en el siguiente paso podemos agregar:

- diagramas conmutativos de axiomas de grupo en **TopAb** usando `tikz-cd`,
- un ejemplo completo de acción de grupo e invariancia (p.ej. traslaciones en imágenes),
- una sección de "categorías para deep learning" (Monoidal categories / string diagrams).